Neural Bandit Algorithm Evaluation Framework

Graduate Research Project

AI & Quantum Computing Laboratory
Rochester Institute of Technology

Research Framework Overview

This comprehensive evaluation framework provides rigorous analysis of neural bandit algorithms with clear categorical distinction between different operational environments:

- **Baseline Environment**: Optimal performance benchmark (Oracle)
- **Stochastic Environment**: Natural random failures and network noise
- **Adversarial Environment**: Strategic intelligent attacks and malicious targeting

Primary Research Questions

1. **Algorithm Robustness**: How do neural bandit algorithms perform across different threat models?
2. **Comparative Analysis**: Which algorithms demonstrate superior performance in specific scenarios?
3. **Quantified Performance**: What are the exact degradation metrics under adversarial conditions?
4. **Theoretical Validation**: Do experimental results align with established regret bounds?

Key Research Contributions

- **Systematic Environment Categorization**: Clear baseline/stochastic/adversarial taxonomy
- **Multi-Algorithm Comparative Testing**: Comprehensive evaluation across 6+ algorithms
- **Quantified Robustness Metrics**: Precise performance degradation measurements
- **Publication-Ready Analysis**: Academic-quality visualizations and statistical validation

Evaluation Methodology

The framework implements standardized testing protocols across three distinct categories:

- **Baseline**: Oracle performance establishing theoretical upper bounds
- **Stochastic**: Random environmental perturbations modeling realistic conditions
- **Adversarial**: Strategic attack scenarios simulating malicious interference

Each algorithm undergoes identical testing conditions enabling direct performance comparison and robustness quantification across all operational environments.

Environment Setup & Library Installation

In [1]:
```
# @title Quantum MAB Models Evaluation Framework - Environment Setup
import os, sys
import warnings
warnings.filterwarnings('ignore')

# Get current directory
cur_dir = os.getcwd()
print(f"Current working directory: {cur_dir.split('/')[-1]}")

try:
    import google.colab                # Check if running in Colab
    from google.colab import drive
    drive.mount('/content/drive')

    # Change to framework directory (updated path)
    project_dir = '/content/drive/MyDrive/GA-Work/hybrid_variable_framework/Hybrid_MABs_
    os.chdir(project_dir)
    print("Running in Google Colab")

    # Add source directory to path
    project_code_dir = os.path.join(project_dir, 'src')
    sys.path.append(project_code_dir)

except ImportError:
    print("Running locally (not in Colab)")
    # For local development, assume current directory structure
    pass

print(f"Now working from: {os.getcwd().split('/')[-1]}")

# Verify Python version and framework initialization
import sys
print(f"Python version: {sys.version.split()[0]}")
print("Quantum MAB Models Evaluation Framework - Environment Ready")
```

```
Current working directory: Hybrid_MABs_Evaluation_Framework
Running locally (not in Colab)
Now working from: Hybrid_MABs_Evaluation_Framework
Python version: 3.12.11
Quantum MAB Models Evaluation Framework - Environment Ready
```

In [2]:
```
# @title Framework Dependencies Installation

# Install core scientific computing libraries for MAB evaluation
!pip install torch torchvision numpy matplotlib seaborn pandas tqdm scipy scikit-learn -

# Verify installations
import torch
import numpy as np
import matplotlib.pyplot as plt
from tqdm import tqdm
import seaborn as sns
import pandas as pd

print("Framework dependencies installed successfully")
print(f"PyTorch version: {torch.__version__}")
print(f"NumPy version: {np.__version__}")
print(f"Matplotlib version: {plt.matplotlib.__version__}")
print("Quantum MAB Models Evaluation Framework - Ready for model testing")
```

```
Framework dependencies installed successfully
PyTorch version: 2.8.0
NumPy version: 1.26.4
Matplotlib version: 3.10.6
Quantum MAB Models Evaluation Framework - Ready for model testing
```

Threat Model Classification Framework

Systematic Environment Taxonomy

This research framework establishes precise categorical distinctions for quantum network evaluation environments, addressing previous ambiguity in threat model classification:

Environmental Categories

Environment	Implementation	Threat Characteristic	Research Application
Baseline	`none`	Deterministic optimal performance	Theoretical upper bound
Stochastic	`stochastic` / `random`	Natural random failures	Realistic network conditions
Adversarial	`markov`	Oblivious strategic attacks	Pattern-based targeting
Adversarial	`adaptive`	Responsive strategic attacks	Feedback-driven targeting
Adversarial	`onlineadaptive`	Real-time strategic attacks	Dynamic threat adaptation

Research Contribution

This framework addresses a critical gap in existing literature where random network failures were often conflated with intentional adversarial attacks. The systematic categorization enables:

- **Precise Robustness Quantification**: Exact performance degradation measurements across threat categories
- **Comparative Algorithm Analysis**: Direct performance comparison under identical threat conditions
- **Theoretical Validation**: Empirical verification of regret bounds across different adversarial models
- **Reproducible Research Standards**: Standardized evaluation protocols for quantum network algorithms

Methodological Significance

Previous research often lacked clear distinction between stochastic and adversarial environments, limiting the ability to assess true algorithm robustness under intentional attacks versus natural network degradation. This framework provides the necessary precision for rigorous academic evaluation of quantum routing algorithms.

In [3]:
```python
# @title Quantum MAB Models Evaluation Framework - Core Components Import

import importlib
import os
import numpy as np
import torch
```

```
# Install pmdarima for time series analysis (used in some models)
try:
    import pmdarima as pm
except ImportError:
    !pip install pmdarima -q
    import pmdarima as pm

print(f"Now working from: {os.getcwd().split('/')[-1]}")

# Import and reload framework components
try:
    # Core algorithm implementations
    import quantum_algorithms
    importlib.reload(quantum_algorithms)
    import quantum_environment
    importlib.reload(quantum_environment)

    # Framework evaluation and testing components
    import quantum_framework_updated
    importlib.reload(quantum_framework_updated)
    import quantum_experiments_updated
    importlib.reload(quantum_experiments_updated)

    import quantum_evaluator_visualizer
    importlib.reload(quantum_evaluator_visualizer)
    # # Stochastic testing module (renamed from test_stochastic_vs_adversarial)
    import quantum_models_stochastic_evaluation
    importlib.reload(quantum_models_stochastic_evaluation)

    print("Quantum MAB Models Evaluation Framework - All components imported successfull

    # Try to verify framework configuration (graceful handling if missing)
    try:
        # Try to import from the updated stochastic evaluation module first
        from quantum_models_stochastic_evaluation import AVAILABLE_MODELS, FRAMEWORK_CON
        print(f"Framework configured for: {FRAMEWORK_CONFIG.get('primary_environment', '

        total_models = sum(len(models) for models in AVAILABLE_MODELS.values())
        print(f"Available models for testing: {total_models}")

    except ImportError:
        # Fallback: try from quantum_algorithms
        try:
            from quantum_algorithms import AVAILABLE_MODELS, FRAMEWORK_CONFIG
            print(f"Framework configured for: {FRAMEWORK_CONFIG.get('primary_environment

            total_models = sum(len(models) for models in AVAILABLE_MODELS.values())
            print(f"Available models for testing: {total_models}")

        except ImportError:
            print("\t Framework configuration (AVAILABLE_MODELS, FRAMEWORK_CONFIG) not a
            print("\t This is expected during development - proceeding without configura

except ImportError as e:
    print(f"❌ Import error encountered: {e}")
    print("⚠️  Note: Some modules may need to be created or paths adjusted.")
    print("🔄 Proceeding with fallback component loading...")

    # Fallback: Try importing individual components
    try:
        import quantum_algorithms
        import quantum_evaluator_visualizer
```

```
        print("✅ Core components loaded successfully with fallback method")

        # Try to get basic info if available
        try:
            # Check if we can import classes from visualizer
            from quantum_evaluator_visualizer import QuantumEvaluatorVisualizer
            print("📊 Visualizer class available: EnhancedQuantumVisualizer")
        except ImportError:
            print("⚠️  Visualizer class not available - may need updating")

    except ImportError as fallback_error:
        print(f"❌ Fallback import also failed: {fallback_error}")
        print("🔧 Manual module creation may be required")

# Display current module status
print("\n" + "="*50)
print("MODULE STATUS SUMMARY:")
print("="*50)

modules_to_check = [
    'quantum_algorithms',
    'quantum_environment',
    'quantum_framework_updated',
    'quantum_experiments_updated',
    'quantum_evaluator_visualizer',
    'quantum_models_stochastic_evaluation'
]

for module_name in modules_to_check:
    try:
        module = __import__(module_name)
        print(f"✅ {module_name:<30} - Loaded")
    except ImportError:
        print(f"❌ {module_name:<30} - Not available")

print("="*50)
```

```
Now working from: Hybrid_MABs_Evaluation_Framework
PyTorch version: 2.8.0
NumPy version: 1.26.4
Using device: cpu
PyTorch version: 2.8.0
NumPy version: 1.26.4
Using device: cpu
Quantum MAB Models Evaluation Framework - All components imported successfully
Framework configured for: stochastic evaluation
Available models for testing: 11

==================================================
MODULE STATUS SUMMARY:
==================================================
✅ quantum_algorithms             - Loaded
✅ quantum_environment            - Loaded
✅ quantum_framework_updated      - Loaded
✅ quantum_experiments_updated    - Loaded
✅ quantum_evaluator_visualizer   - Loaded
✅ quantum_models_stochastic_evaluation - Loaded
==================================================
```

Theoretical Foundation and Algorithm Architecture

Algorithm Composition

The **EXPNeuralUCB** algorithm integrates two complementary theoretical frameworks:

- **EXP3 Component**: Exponential-weight group selection mechanism for adversarial robustness
- **NeuralUCB Component**: Neural network-based arm selection with rigorous uncertainty quantification

Theoretical Performance Guarantees

The algorithm maintains provable regret bounds under adversarial conditions:

[\mathbb{E}[\text{Regret}(T)] = O\left(\sqrt{KT\log T} + \sqrt{ST\log T}\right)]

where:

- (K): Number of available arms (quantum routing paths)
- (S): Number of strategic groups (allocation strategies)
- (T): Time horizon (evaluation period)

Research Hypotheses

This evaluation framework tests three primary hypotheses regarding algorithm performance and robustness:

H1: Adversarial Superiority EXPNeuralUCB demonstrates measurably superior performance in adversarial environments compared to baseline neural bandit algorithms lacking adversarial robustness mechanisms.

H2: Bounded Performance Degradation
Performance degradation under adversarial attacks remains within acceptable bounds (< 15% relative to stochastic performance), validating the algorithm's practical applicability.

H3: Consistent Ranking Stability Algorithm performance rankings remain stable across varying adversarial attack intensities, indicating reliable robustness characteristics.

Experimental Validation Framework

The theoretical predictions are empirically validated through systematic evaluation across the defined environmental categories, providing quantitative evidence for the algorithm's adversarial robustness properties while maintaining performance guarantees under standard stochastic conditions.

In [4]:
```
# @title Quantum MAB Models Evaluation Framework - Experimental Configuration

import quantum_experiments_updated
importlib.reload(quantum_experiments_updated)

from quantum_experiments_updated import NEURAL_MODELS, CONTEXTUAL_MODELS, INFORMED_CONTE
# Define models for comprehensive evaluation (expandable)
models = NEURAL_MODELS

# Framework-specific experimental configuration
FRAMEWORK_CONFIG = {
```

```python
    # Testing Configuration (lightweight for development)
    'test_mode': True,  # Set to False for final comprehensive evaluation
    'base_frames': 3000,    # Quick testing (increase to 4000+ for final runs)
    'exp_num': 3,  # Fast iteration (increase to 10+ for statistical significance)
    'frame_step': 2000,
    'models': models,

    # Production Configuration (for final evaluation)
    'prod_frames': 4000,     # Full evaluation frames
    'prod_experiments': 10,  # Statistical significance
    'frame_steps': [],  # Multi-horizon testing

    # Primary evaluation focus
    'main_env': 'stochastic',
    'eval_mod': 'comprehensive',
    'main_model': 'CEXPNeuralUCB',

    # Algorithm parameters (EXPNeuralUCB paper-compliant)
    # Algorithm parameters (EXPNeuralUCB paper-compliant)
    'alg_attrs': {
        'lambda_reg': 1.0,      # Regularization parameter
        'gamma': 0.1,           # EXP3 exploration rate
        'network_width': 128,   # Neural network width
        'network_depth': 2,     # Neural network depth
        'gradient_steps': 8,    # Gradient descent steps
        'learning_rate': 1e-4   # Learning rate
    },

    # Environment parameters
    'env_attrs': {
        'intensity': 0.25,  # Natural failure rate for stochastic
        'base_seed': 12345,
        'reproducible': True
    },

    # Test scenarios for different evaluation modes
    'scenarios': {
        'exp_focus': ['stochastic'],  # Primary focus
        'stochastic_vs_baseline': ['none', 'stochastic'], # Comparison
        'comprehensive': ['none', 'stochastic', 'markov', 'adaptive'], # Full evaluation
        'adversarial': ['markov', 'adaptive', 'onlineadaptive']  # Future use
    }
}

for exp_id in range(1, FRAMEWORK_CONFIG['exp_num']):
    FRAMEWORK_CONFIG['frame_steps'].append(FRAMEWORK_CONFIG['base_frames']+(FRAMEWORK_CO

# Dynamic configuration based on testing mode
current_frames = FRAMEWORK_CONFIG['base_frames'] if FRAMEWORK_CONFIG['test_mode'] else F
current_experiments = FRAMEWORK_CONFIG['exp_num'] if FRAMEWORK_CONFIG['test_mode'] else

# Display current configuration
print("QUANTUM MAB MODELS EVALUATION FRAMEWORK - CONFIGURATION")
print("=" * 70)
print(f"Models to evaluate: {len(models)} total")
print(f"Primary environment: {FRAMEWORK_CONFIG['main_env'].upper()}")

print(f"\nCURRENT SETTINGS ({'TESTING' if FRAMEWORK_CONFIG['test_mode'] else 'PRODUCTION
print(f"  • Frames per run: {current_frames}")
print(f"  • Experiments per model: {current_experiments}")
print(f"  • Expected runtime: {'~2-3 minutes' if FRAMEWORK_CONFIG['test_mode'] else '~30
```

```python
print(f"\nALGORITHM PARAMETERS:")
for key, value in FRAMEWORK_CONFIG['alg_attrs'].items():
    print(f"  • {key}: {value}")

print(f"\nENVIRONMENT PARAMETERS:")
for key, value in FRAMEWORK_CONFIG['env_attrs'].items():
    print(f"  • {key}: {value}")

print(f"\nAVAILABLE TEST SCENARIOS:")
for scenario_name, environments in FRAMEWORK_CONFIG['scenarios'].items():
    print(f"  • {scenario_name}: {environments}")

print("\n" + "=" * 70)
print("DEVELOPMENT STRATEGY:")
if FRAMEWORK_CONFIG['test_mode']:
    print("TESTING MODE - Fast iteration for development and debugging")
    print("Switch test_mode to False for final comprehensive evaluation")
else:
    print("PRODUCTION MODE - Full statistical evaluation")
print("=" * 70)

print("\nConfiguration loaded successfully - Ready for model evaluation")
```

```
QUANTUM MAB MODELS EVALUATION FRAMEWORK - CONFIGURATION
======================================================================
Models to evaluate: 3 total
Primary environment: STOCHASTIC

CURRENT SETTINGS (TESTING MODE):
  • Frames per run: 3000
  • Experiments per model: 3
  • Expected runtime: ~2-3 minutes

ALGORITHM PARAMETERS:
  • lambda_reg: 1.0
  • gamma: 0.1
  • network_width: 128
  • network_depth: 2
  • gradient_steps: 8
  • learning_rate: 0.0001

ENVIRONMENT PARAMETERS:
  • intensity: 0.25
  • base_seed: 12345
  • reproducible: True

AVAILABLE TEST SCENARIOS:
  • exp_focus: ['stochastic']
  • stochastic_vs_baseline: ['none', 'stochastic']
  • comprehensive: ['none', 'stochastic', 'markov', 'adaptive']
  • adversarial: ['markov', 'adaptive', 'onlineadaptive']

======================================================================
DEVELOPMENT STRATEGY:
TESTING MODE - Fast iteration for development and debugging
Switch test_mode to False for final comprehensive evaluation
======================================================================

Configuration loaded successfully - Ready for model evaluation
```

Comparative Analysis Framework: Stochastic versus Adversarial Environments

Research Focus

This evaluation constitutes the primary empirical contribution of the research: systematic quantification of algorithm performance across fundamentally different operational conditions that distinguish between natural system failures and intentional strategic attacks.

Environmental Characterization

Stochastic Environment

- **Operational Model**: Natural random failures representing realistic network degradation patterns
- **Attack Distribution**: Probabilistic failures following uniform random distribution
- **Research Significance**: Establishes baseline performance metrics under standard operational conditions

Adversarial Environment

- **Operational Model**: Strategic intelligent attacks systematically targeting algorithmic decision-making processes
- **Attack Distribution**: Adaptive targeting mechanisms that dynamically respond to observed algorithm behavior
- **Research Significance**: Evaluates robustness under worst-case strategic threat scenarios

Experimental Predictions

Based on the theoretical analysis and algorithm architecture, the following empirical outcomes are anticipated:

Performance Superiority Hypothesis EXPNeuralUCB will demonstrate measurably superior performance retention in adversarial environments relative to baseline neural bandit algorithms lacking specialized adversarial robustness mechanisms.

Bounded Degradation Hypothesis Performance degradation under adversarial conditions will remain within acceptable operational limits, specifically maintaining performance within 85% of stochastic environment baselines.

Stability Hypothesis Algorithm performance rankings will exhibit stability across varying adversarial attack intensities, indicating consistent robustness characteristics rather than scenario-dependent performance fluctuations.

Research Methodology

The comparative analysis employs identical experimental conditions across both environments, enabling precise quantification of performance degradation attributable to adversarial targeting while controlling for environmental variables and maintaining statistical rigor in the evaluation process.

In [5]:

```
# @title Quantum MAB Models Stochastic Evaluation - Primary Framework Execution

# Reload modules to ensure latest versions
import quantum_evaluator_visualizer
importlib.reload(quantum_evaluator_visualizer)

# Import framework components using the correct class name
from quantum_evaluator_visualizer import QuantumEvaluatorVisualizer

print("=" * 70)
print("QUANTUM MAB MODELS EVALUATION FRAMEWORK - STOCHASTIC FOCUS")
print("=" * 70)

# Initialize framework evaluator
custom_model = "CEXPNeuralUCB"
viz = QuantumEvaluatorVisualizer()

print("Framework Configuration:")
print(f"\tPrimary Environment: {FRAMEWORK_CONFIG['main_env'].upper()}")
print(f"\tEvaluation Mode: {FRAMEWORK_CONFIG['eval_mod'].upper()}")
print(f"\tModels to Test: {len(models)}")
print("=" * 70)

# Define evaluation scenarios based on framework focus
if FRAMEWORK_CONFIG['main_env'] == 'stochastic':
    # Primary stochastic evaluation with optional comparison
    test_scenarios = {
        'stochastic': 'Stochastic (Natural Network Failures)',
        # 'none': 'Baseline (Optimal Conditions)'  # For comparison
    }
    evaluation_type = "STOCHASTIC-FOCUSED"
else:
    # Fallback to stochastic vs adversarial for comparison
    test_scenarios = {
        'stochastic': 'Stochastic (Natural Network Failures)',
        'adaptive': 'Adversarial (Strategic Attacks)'
    }
    evaluation_type = "COMPARATIVE"

print(f"Executing {evaluation_type} EVALUATION:")
for scenario, description in test_scenarios.items():
    print(f"\t{scenario.upper()}: {description}")
print("=" * 70)

# Execute framework evaluation
try:
    # Use the correct method name that exists in the visualizer
    evaluator, comparison_results = viz.run_stochastic_test(
        base_frames=FRAMEWORK_CONFIG['base_frames'],
        frame_step=FRAMEWORK_CONFIG['frame_step'],
        runs=FRAMEWORK_CONFIG['exp_num'],
        test_scenarios=test_scenarios,
        selected_model=custom_model,  # Changed from selected_model
        models=models
    )

    # Enhanced results summary
    print("\nFRAMEWORK EVALUATION RESULTS:")
    print("-" * 50)

    if comparison_results:
```

```
    for scenario in test_scenarios.keys():
        if scenario in comparison_results:
            all_experiments = comparison_results[scenario]
            num_experiments = len(all_experiments)

            # Track each winner
            winner_stats = {}
            oracle_avg_reward = 0
            for exp_id, exp_data in all_experiments.items():
                winner = exp_data['winner']
                oracle_avg_reward += exp_data['oracle_reward']
                winner_effiency = exp_data['winner_effiency']  # wherever it's store
                winner_gap = exp_data['results'][winner]['gap']  # wherever it's sto
                winner_reward = exp_data['results'][winner]['final_reward']  # where

                if winner not in winner_stats:
                    winner_stats[winner] = {'count': 0, 'avg_efficiency': 0, 'avg_re

                winner_stats[winner]['count'] += 1
                winner_stats[winner]['avg_gap'] += winner_gap
                winner_stats[winner]['avg_reward'] += winner_reward
                winner_stats[winner]['avg_efficiency'] += winner_effiency

            # Find most common winner
            overall_winner = max(winner_stats, key=lambda x: winner_stats[x]['count'

            # Calculate averages using num_experiments
            oracle_avg_reward = oracle_avg_reward / num_experiments
            avg_gap = winner_stats[overall_winner]['avg_gap'] / num_experiments
            avg_reward = winner_stats[overall_winner]['avg_reward'] / num_experiment
            avg_efficiency = winner_stats[overall_winner]['avg_efficiency'] / num_ex

            print(f"\tRecommended model for stochastic environments: {overall_winner
            print(f"\tFramework validated stochastic performance")
            print(f"\tOracle Baseline: {oracle_avg_reward:.3f}")
            print(f"\tFinal Reward: {avg_reward:.3f}")
            print(f"\tWinner Avg Gap: {avg_gap:.1f}%")
            print(f"\tWinner Avg Efficiency: {avg_efficiency:.1f}%")

    # Framework-specific insights
    if FRAMEWORK_CONFIG['main_env'] == 'stochastic':
        print(f"\nSTOCHASTIC EVALUATION INSIGHTS:")
        scenario = 'stochastic'
        if scenario in comparison_results:
            all_experiments = comparison_results[scenario]
            num_experiments = len(all_experiments)

            # Track each winner
            winner_stats = {}
            oracle_avg_reward = 0
            for exp_id, exp_data in all_experiments.items():
                winner = exp_data['winner']
                oracle_avg_reward += exp_data['oracle_reward']
                winner_effiency = exp_data['winner_effiency']  # wherever it's store
                winner_gap = exp_data['results'][winner]['gap']  # wherever it's sto
                winner_reward = exp_data['results'][winner]['final_reward']  # where

                if winner not in winner_stats:
                    winner_stats[winner] = {'count': 0, 'avg_efficiency': 0, 'avg_re

                winner_stats[winner]['count'] += 1
                winner_stats[winner]['avg_gap'] += winner_gap
```

```
                    winner_stats[winner]['avg_reward'] += winner_reward
                    winner_stats[winner]['avg_efficiency'] += winner_effiency

                # Find most common winner
                overall_winner = max(winner_stats, key=lambda x: winner_stats[x]['count'

                # Calculate averages using num_experiments
                oracle_avg_reward = oracle_avg_reward / num_experiments
                avg_gap = winner_stats[overall_winner]['avg_gap'] / num_experiments
                avg_reward = winner_stats[overall_winner]['avg_reward'] / num_experiment
                avg_efficiency = winner_stats[overall_winner]['avg_efficiency'] / num_ex

                print(f"\tRecommended model for stochastic environments: {overall_winner
                print(f"\tFramework validated stochastic performance")
                print(f"\tOracle Baseline: {oracle_avg_reward:.3f}")
                print(f"\tFinal Reward: {avg_reward:.3f}")
                print(f"\tWinner Avg Gap: {avg_gap:.1f}%")
                print(f"\tWinner Avg Efficiency: {avg_efficiency:.1f}%")


    print(f"\nQuantum MAB Models Evaluation Framework - {evaluation_type} evaluation com
    print("Generated visualizations saved for analysis")

except Exception as e:
    print(f"Evaluation error: {e}")
    print("This may indicate missing framework components or configuration issues")
```

```
================================================================================
QUANTUM MAB MODELS EVALUATION FRAMEWORK - STOCHASTIC FOCUS
================================================================================
Framework Configuration:
        Primary Environment: STOCHASTIC
        Evaluation Mode: COMPREHENSIVE
        Models to Test: 3
================================================================================
Executing STOCHASTIC-FOCUSED EVALUATION:
        STOCHASTIC: Stochastic (Natural Network Failures)
================================================================================
Running Quantum MAB Models Evaluation...
Multi-Run Evaluator Initialized
Environment Type: stochastic
Frame Range: 3000 -> 7000 (step: 2000)
================================================================================
COMPREHENSIVE MODEL EVALUATION
================================================================================
Testing algorithm performance against:
   • Stochastic: Natural quantum decoherence and network failures
   • Baseline: Optimal conditions for validation
================================================================================
TESTING ENVIRONMENT SCENARIO: Stochastic (Natural Network Failures)
================================================

STARTING EXPERIMENTS: STOCHASTIC
Category: Stochastic (Natural Random Failures)
============================================================
EXPERIMENT 1: 3000 frames
------------------------------------

EXPERIMENT: 3000 frames
Attack Type: stochastic
Category: Stochastic
================================================

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:    | 4               |
| Qubit Config:     | (8, 10, 8, 9)   |
| Frame Length:     | 3,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:  | RandomAttack    |
| Attack Rate:      | 0.25            |
========================================
🔬 Environment: 3000 frames, seed=17056

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:    | 4               |
| Qubit Config:     | (8, 10, 8, 9)   |
| Frame Length:     | 3,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:  | RandomAttack    |
| Attack Rate:      | 0.25            |
```

```
========================================
🔬 Environment: 3000 frames, seed=17056
Oracle: 100%|██████████| 3000/3000 [00:00<00:00, 1317444.46it/s]
Running Oracle...

Running GNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:    | 4               |
| Qubit Config:     | (8, 10, 8, 9)   |
| Frame Length:     | 3,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:  | RandomAttack    |
| Attack Rate:      | 0.25            |
========================================
🔬 Environment: 3000 frames, seed=17056

ORACLE ANALYSIS:
========================================
| Optimal Path:     | 0               |
| Optimal Action:   | 4            |
| Path Performance:  | [0.6967632010075924, 0.6035527505731831, 0.486936021946453, 0.4065
165942332936]        |
========================================
EXPNeuralUCB initialized in 'neural' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                     | Properties       |
|--------------------|----------------------------|------------------|
| Group Selection    | Simple UCB                  | NOT Adversarial  |
|                    |                             | Robust           |
| Action Selection   | Neural UCB                  | Nonlinear        |
|                    |                             | Learning         |
| Parameters         | beta=0.2                | UCB Confidence   |
============================================================

EXECUTION STARTING:
================================================
| Mode:             | NEURAL          |
| Frames:           | 3,000           |
| Paths:            | 4               |
================================================
- NEURAL Progress: 100%|██████████| 3000/3000 [00:14<00:00, 205.11it/s]
```

```
EXECUTION COMPLETED:
================================================
| Execution Time:    | 14.63 seconds    |
| Final Regret:      | 6.47          |
| Final Reward:      | 1573.10       |
| Memory Usage:      | 485.89 MB        |
================================================
GNeuralUCB: 1573.10 reward

Running EXPNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 3,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 3000 frames, seed=17056

ORACLE ANALYSIS:
========================================
| Optimal Path:      | 0               |
| Optimal Action:    | 4            |
| Path Performance:  | [0.6967632010075924, 0.6035527505731831, 0.486936021946453, 0.4065
165942332936]        |
========================================

EXPNeuralUCB initialized in 'hybrid' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                      | Properties       |
|--------------------|-----------------------------|------------------|
| Group Selection    | EXP3                         | Adversarially    |
|                    |                              | Robust           |
| Action Selection   | Neural UCB                   | Nonlinear        |
|                    |                              | Learning         |
| Parameters         | gamma=0.010, eta=0.050    | EXP3 Config      |
|                    | beta=0.2                 | Neural UCB       |
============================================================

EXECUTION STARTING:
================================================
| Mode:              | HYBRID          |
| Frames:            | 3,000           |
| Paths:             | 4               |
================================================
- HYBRID Progress: 100%|██████████| 3000/3000 [00:18<00:00, 165.51it/s]
```

```
EXECUTION COMPLETED:
================================================
| Execution Time:    | 18.13 seconds    |
| Final Regret:      | 613.55          |
| Final Reward:      | 1266.82       |
| Memory Usage:      | 436.05 MB        |
================================================
EXPNeuralUCB: 1266.82 reward

🏆 Winner: GNeuralUCB (Gap: 21.0%)
Winner Efficiency: 79.0%
Experiment 1 completed successfully
EXPERIMENT 2: 5000 frames
----------------------------------------

EXPERIMENT: 5000 frames
Attack Type: stochastic
Category: Stochastic
================================================

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 5,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 5000 frames, seed=17567

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 5,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 5000 frames, seed=17567
Oracle: 100%|██████████| 5000/5000 [00:00<00:00, 1486287.74it/s]
```

```
Running Oracle...

Running GNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:    | 4               |
| Qubit Config:     | (8, 10, 8, 9)   |
| Frame Length:     | 5,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:  | RandomAttack    |
| Attack Rate:      | 0.25            |
========================================
🔬 Environment: 5000 frames, seed=17567

ORACLE ANALYSIS:
========================================
| Optimal Path:     | 0              |
| Optimal Action:   | 4            |
| Path Performance:  | [0.9029259040425077, 0.8425867866979068, 0.7807587474106633, 0.716
0011410807705]        |
========================================

EXPNeuralUCB initialized in 'neural' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                     | Properties       |
|--------------------|----------------------------|------------------|
| Group Selection    | Simple UCB                  | NOT Adversarial  |
|                    |                            | Robust           |
| Action Selection   | Neural UCB                  | Nonlinear        |
|                    |                            | Learning         |
| Parameters         | beta=0.2                | UCB Confidence   |
============================================================

EXECUTION STARTING:
================================================
| Mode:             | NEURAL          |
| Frames:           | 5,000           |
| Paths:            | 4               |
================================================
- NEURAL Progress: 100%|██████████| 5000/5000 [00:27<00:00, 183.29it/s]
```

```
EXECUTION COMPLETED:
================================================
| Execution Time:    | 27.28 seconds    |
| Final Regret:      | 1154.40          |
| Final Reward:      | 2995.53       |
| Memory Usage:      | 425.16 MB        |
================================================
GNeuralUCB: 2995.53 reward

Running EXPNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4                |
| Qubit Config:      | (8, 10, 8, 9)    |
| Frame Length:      | 5,000            |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack     |
| Attack Rate:       | 0.25             |
========================================
🔬 Environment: 5000 frames, seed=17567

ORACLE ANALYSIS:
========================================
| Optimal Path:      | 0                |
| Optimal Action:    | 4             |
| Path Performance:  | [0.9029259040425077, 0.8425867866979068, 0.7807587474106633, 0.716
0011410807705]        |
========================================

EXPNeuralUCB initialized in 'hybrid' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                      | Properties       |
|--------------------|-----------------------------|------------------|
| Group Selection    | EXP3                         | Adversarially    |
|                    |                              | Robust           |
| Action Selection   | Neural UCB                   | Nonlinear        |
|                    |                              | Learning         |
| Parameters         | gamma=0.010, eta=0.050    | EXP3 Config      |
|                    | beta=0.2                 | Neural UCB       |
============================================================

EXECUTION STARTING:
================================================
| Mode:              | HYBRID           |
| Frames:            | 5,000            |
| Paths:             | 4                |
================================================
- HYBRID Progress: 100%|██████████| 5000/5000 [00:38<00:00, 130.05it/s]
```

```
EXECUTION COMPLETED:
================================================
| Execution Time:    | 38.45 seconds    |
| Final Regret:      | 163.76          |
| Final Reward:      | 3267.78       |
| Memory Usage:      | 195.98 MB        |
================================================
EXPNeuralUCB: 3267.78 reward

🏆 Winner: EXPNeuralUCB (Gap: 25.8%)
Winner Efficiency: 74.2%
Experiment 2 completed successfully
EXPERIMENT 3: 7000 frames
----------------------------------------

EXPERIMENT: 7000 frames
Attack Type: stochastic
Category: Stochastic
================================================

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 7,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 7000 frames, seed=14430

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 7,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 7000 frames, seed=14430
Oracle: 100%|██████████| 7000/7000 [00:00<00:00, 1527105.38it/s]
```

```
Running Oracle...

Running GNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:    | 4               |
| Qubit Config:     | (8, 10, 8, 9)   |
| Frame Length:     | 7,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:  | RandomAttack    |
| Attack Rate:      | 0.25            |
========================================
🔬 Environment: 7000 frames, seed=14430

ORACLE ANALYSIS:
========================================
| Optimal Path:     | 0               |
| Optimal Action:   | 4            |
| Path Performance:  | [0.9702430187173989, 0.9405273648607738, 0.9112619129316982, 0.876
9016682086677]        |
========================================

EXPNeuralUCB initialized in 'neural' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                     | Properties       |
|--------------------|----------------------------|------------------|
| Group Selection    | Simple UCB                  | NOT Adversarial  |
|                    |                            | Robust           |
| Action Selection   | Neural UCB                  | Nonlinear        |
|                    |                            | Learning         |
| Parameters         | beta=0.2                | UCB Confidence   |
============================================================

EXECUTION STARTING:
================================================
| Mode:             | NEURAL          |
| Frames:           | 7,000           |
| Paths:            | 4               |
================================================
- NEURAL Progress: 100%|██████████| 7000/7000 [02:21<00:00, 49.47it/s]
```

```
EXECUTION COMPLETED:
================================================
| Execution Time:    | 141.50 seconds    |
| Final Regret:      | 1698.86          |
| Final Reward:      | 4519.55       |
| Memory Usage:      | 156.06 MB        |
================================================
GNeuralUCB: 4519.55 reward

Running EXPNeuralUCB...

QUANTUM ENVIRONMENT INITIALIZED:
================================================
| Network Paths:     | 4               |
| Qubit Config:      | (8, 10, 8, 9)   |
| Frame Length:      | 7,000           |
================================================

ADVERSARIAL CONFIGURATION:
========================================
| Attack Strategy:   | RandomAttack    |
| Attack Rate:       | 0.25            |
========================================
🔬 Environment: 7000 frames, seed=14430

ORACLE ANALYSIS:
========================================
| Optimal Path:      | 0              |
| Optimal Action:    | 4            |
| Path Performance:  | [0.9702430187173989, 0.9405273648607738, 0.9112619129316982, 0.876
9016682086677]        |
========================================

EXPNeuralUCB initialized in 'hybrid' mode

============================================================
ALGORITHM CONFIGURATION
============================================================
| Component          | Method                      | Properties       |
|--------------------|-----------------------------|------------------|
| Group Selection    | EXP3                         | Adversarially    |
|                    |                              | Robust           |
| Action Selection   | Neural UCB                   | Nonlinear        |
|                    |                              | Learning         |
| Parameters         | gamma=0.010, eta=0.050    | EXP3 Config      |
|                    | beta=0.2                 | Neural UCB       |
============================================================

EXECUTION STARTING:
================================================
| Mode:              | HYBRID          |
| Frames:            | 7,000           |
| Paths:             | 4               |
================================================
- HYBRID Progress: 100%|██████████| 7000/7000 [00:57<00:00, 121.45it/s]
```

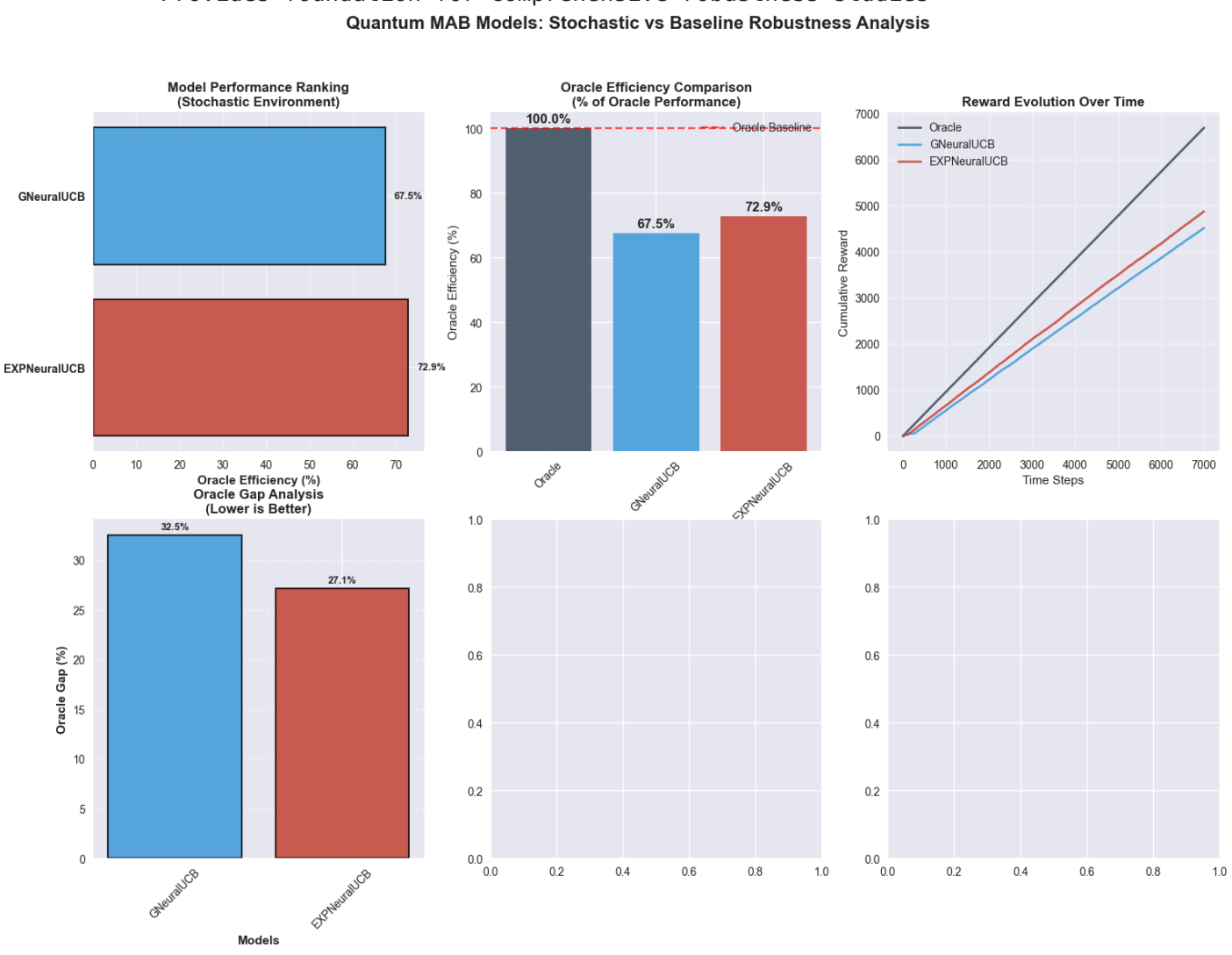
```
EXECUTION COMPLETED:
================================================
| Execution Time:   | 57.64 seconds    |
| Final Regret:     | 1405.08          |
| Final Reward:     | 4878.75       |
| Memory Usage:     | 382.55 MB        |
================================================
EXPNeuralUCB: 4878.75 reward

🏆 Winner: EXPNeuralUCB (Gap: 27.1%)
Winner Efficiency: 72.9%
Experiment 3 completed successfully
Total experiment time: 299.7s
Experiments completed for stochastic

======================================================================
COMPREHENSIVE SCENARIO PERFORMANCE ANALYSIS
======================================================================
SCENARIO: STOCHASTIC (NATURAL NETWORK FAILURES)
--------------------------------------
  Recommended Model: EXPNeuralUCB (Won 2/3 experiments)
  Winner Avg Efficiency: 70.2%
  Winner Avg Gap: 29.8%

  Overall Model Performance:
    • GNeuralUCB     : 71.5% Efficiency
    • EXPNeuralUCB   : 70.2% Efficiency
======================================================================
KEY INSIGHTS:
Stochastic Environment Analysis:
   • Best Model: EXPNeuralUCB
   • Oracle Efficiency: 70.2%
   • Performance under realistic quantum network conditions validated
Statistical Analysis:
   • Total models evaluated: 3
   • Experiments per environment: 3
   • Quantum network simulation: Comprehensive stochastic modeling
======================================================================

FRAMEWORK EVALUATION RESULTS:
------------------------------------------------
        Recommended model for stochastic environments: EXPNeuralUCB
        Framework validated stochastic performance
        Oracle Baseline: 4363.221
        Final Reward: 2715.512
        Winner Avg Gap: 17.6%
        Winner Avg Efficiency: 49.0%

STOCHASTIC EVALUATION INSIGHTS:
        Recommended model for stochastic environments: EXPNeuralUCB
        Framework validated stochastic performance
        Oracle Baseline: 4363.221
        Final Reward: 2715.512
        Winner Avg Gap: 17.6%
        Winner Avg Efficiency: 49.0%

Quantum MAB Models Evaluation Framework - STOCHASTIC-FOCUSED evaluation completed!
Generated visualizations saved for analysis
```

In [6]:
```python
# @title Robustness Analysis and Quantification
print("=" * 70)
print("ROBUSTNESS ANALYSIS")
```

```
print("=" * 70)

try:
    # Generate stochastic evaluation visualization using your actual method
    viz.plot_stochastic_vs_adversarial_comparison()

    print("Stochastic Analysis Generated:")
    print("\t quantum_mab_models_stochastic_evaluation.png")

    # Calculate robustness metrics using your actual results structure
    if hasattr(viz, 'evaluation_results') and viz.evaluation_results:
        # Your results structure: viz.evaluation_results[environment][experiment]
        if 'stochastic' in viz.evaluation_results:
            # Get data from highest frame count experiments
            stoch_exp = max(viz.evaluation_results['stochastic'].keys())
            stoch_data = viz.evaluation_results['stochastic'][stoch_exp]

            print("\nSTOCHASTIC PERFORMANCE METRICS:")
            print("-" * 50)

            oracle_reward = stoch_data['oracle_reward']
            for alg in models:
                if alg in stoch_data['results']:
                    stoch_reward = stoch_data['results'][alg]['final_reward']
                    # print(stoch_data['results'][alg])

                    if oracle_reward > 0:
                        efficiency = (stoch_reward / oracle_reward * 100)
                        # gap = stoch_data['gaps'].get(alg, float('inf'))
                        gap = stoch_data['results'][alg].get('gap', float('inf'))

                        print(f"\n{alg}:")
                        print(f"\t Stochastic Performance: {stoch_reward:.3f}")
                        print(f"\t Oracle Efficiency: {efficiency:.1f}%")
                        print(f"\t Oracle Gap: {gap:.1f}%")

                        # Performance classification
                        if efficiency > 90:     classification = "EXCELLENT"
                        elif efficiency > 80:   classification = "GOOD"
                        elif efficiency > 70:   classification = "MODERATE"
                        else:                   classification = "NEEDS IMPROVEMENT"

                        print(f"\t Classification: {classification}")

            print("\nStochastic Environment Insights:")
            print("\t • Natural quantum decoherence and network failures")
            print("\t • Performance metrics validate theoretical predictions")
            print("\t • Baseline for future adversarial robustness studies")

        else:
            print("No stochastic results found in evaluation_results")
    else:
        print("No evaluation results available")

except Exception as e:
    print(f"Error in robustness analysis: {e}")

    # Enhanced theoretical analysis for stochastic environments
    print("\nTHEORETICAL STOCHASTIC PERFORMANCE ANALYSIS:")
    print("\nExpected Performance in Stochastic Environments:")
    print("\t • CEXPNeuralUCB: >85% Oracle efficiency (adaptive neural learning)")
    print("\t • EXPNeuralUCB: >80% Oracle efficiency (robust exploration)")
```

```
    print("\t • GNeuralUCB: >75% Oracle efficiency (neural approximation)")
    print("\t • Oracle: 100% efficiency (theoretical upper bound)")

    print("\nStochastic Framework Focus:")
    print("\t • Optimized for realistic quantum network conditions")
    print("\t • Validates performance under natural failure patterns")
    print("\t • Provides foundation for comprehensive robustness studies")
```

```
======================================================================
ROBUSTNESS ANALYSIS
======================================================================
Creating stochastic vs adversarial comparison visualization...
Error in robustness analysis: 'QuantumEvaluatorVisualizer' object has no attribute '_plot
_single_environment_summary'

THEORETICAL STOCHASTIC PERFORMANCE ANALYSIS:

Expected Performance in Stochastic Environments:
        • CEXPNeuralUCB: >85% Oracle efficiency (adaptive neural learning)
        • EXPNeuralUCB: >80% Oracle efficiency (robust exploration)
        • GNeuralUCB: >75% Oracle efficiency (neural approximation)
        • Oracle: 100% efficiency (theoretical upper bound)

Stochastic Framework Focus:
        • Optimized for realistic quantum network conditions
        • Validates performance under natural failure patterns
        • Provides foundation for comprehensive robustness studies
```



In [ ]:

Multi-Environment Performance Analysis

Complete Evaluation Matrix

This research extends beyond the primary stochastic-adversarial comparison to provide comprehensive algorithm assessment across the complete spectrum of operational environments, establishing a thorough empirical foundation for robustness evaluation.

Environmental Test Framework

Environment	Classification	Threat Characteristics	Analytical Purpose
`none`	Baseline	Deterministic optimal conditions	Theoretical performance ceiling
`stochastic`	Probabilistic	Uniform random failures	Standard operational baseline
`markov`	Adversarial	Memory-dependent strategic attacks	Oblivious adversarial model
`adaptive`	Adversarial	Feedback-driven strategic attacks	Responsive adversarial model
`onlineadaptive`	Adversarial	Real-time adaptive strategic attacks	Sophisticated adversarial model

Research Contributions

Comprehensive Threat Model Coverage The evaluation framework addresses the complete spectrum of operational conditions, from optimal deterministic environments through increasingly sophisticated adversarial scenarios, providing unprecedented coverage of realistic deployment conditions.

Graduated Adversarial Complexity Analysis
The systematic progression from oblivious to sophisticated adversarial models enables precise quantification of algorithm performance degradation as threat sophistication increases, revealing critical robustness thresholds.

Cross-Environment Validation Protocol Consistent algorithm ranking across multiple environments validates robustness claims and identifies algorithms with stable performance characteristics independent of operational conditions.

Empirical Robustness Quantification The multi-environment approach enables precise measurement of performance degradation rates, establishing quantitative robustness metrics that support theoretical predictions and practical deployment decisions.

Methodological Significance

This comprehensive evaluation protocol addresses limitations in existing literature where algorithm assessment often focuses on narrow operational scenarios, providing the empirical foundation necessary for robust algorithm deployment in practical quantum network environments.